12) Blink an LED light using Wi-Fi using Arduino on Cloud

I. thingProperties.h

```
#include <ArduinoIoTCloud.h>
#include <Arduino_ConnectionHandler.h>
const char DEVICE_LOGIN_NAME[]  = "2c7e41f5-f50a-4a88-a65a-90b5f240e56e";

const char SSID[]               = SECRET_SSID;    // Network SSID (name)
const char PASS[]               = SECRET_OPTIONAL_PASS;    // Network password (use for
WPA, or use as key for WEP)
const char DEVICE_KEY[]  = SECRET_DEVICE_KEY;    // Secret device password
void onLedChange();
bool led;

void initProperties(){

  ArduinoCloud.setBoardId(DEVICE_LOGIN_NAME);
  ArduinoCloud.setSecretDeviceKey(DEVICE_KEY);
  ArduinoCloud.addProperty(led, READWRITE, ON_CHANGE, onLedChange);
}
WiFiConnectionHandler ArduinoIoTPreferredConnection(SSID, PASS);
```

II. My_First_Project

```
#include "thingProperties.h"

int ledPin = 13;

void setup() {

  Serial.begin(9600);

  pinMode(ledPin, OUTPUT);   // fixed case

  // Defined in thingProperties.h

  initProperties();

  // Connect to Arduino IoT Cloud

  ArduinoCloud.begin(ArduinoIoTPreferredConnection);

  setDebugMessageLevel(2);

  ArduinoCloud.printDebugInfo();

}

void loop() {

  ArduinoCloud.update();
```

```
  if (led) {   // true means HIGH
    digitalWrite(ledPin, HIGH);
    Serial.println("Led HIGH");
  } else {
    digitalWrite(ledPin, LOW);
    Serial.println("Led LOW");
  }
}
  Since led is READ_WRITE variable, onLedChange() is
    executed every time a new value is received from IoT Cloud.
*/
void onLedChange()  {
  // Add your code here to act upon Led change
}
```

13) Blink an LED light using NodeMCU

```
int ledPin = D4;
void setup()
{
  pinMode(ledPin, OUTPUT); // Set pin as output
}
void loop()
{
  digitalWrite(ledPin, LOW);
  delay(1000);
  digitalWrite(ledPin, HIGH);
  delay(1000);
}
```

14) Experiment with Arduino board interface with UART- Software serial communication.

III. **Code for Sender Arduino**

```
#include <SoftwareSerial.h>
// Define RX and TX pins for software serial
SoftwareSerial mySerial(10, 11); // RX = 10, TX = 11
void setup() {
  Serial.begin(9600);       // Hardware Serial (USB)
  mySerial.begin(9600);     // Software Serial
  Serial.println("Sender Ready...");
 }
void loop() {
  mySerial.println("Hello from Arduino A");  // Send message
  Serial.println("Data Sent: Hello from Arduino A");
  delay(1000);
 }
```

IV. **Code for Receiver Arduino**

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(10, 11); // RX = 10, TX = 11
void setup()
{
  Serial.begin(9600);       // For monitoring
  mySerial.begin(9600);     // For receiving
  Serial.println("Receiver Ready...");
}
void loop() {
  if (mySerial.available())
 {
    String data = mySerial.readStringUntil('\n');
    Serial.print("Received: ");
    Serial.println(data);
  }
}
```

15) Experiment with the Arduino board for software Serial communication for the Bluetooth Module

```
#include <SoftwareSerial.h>
// Define RX and TX pins
SoftwareSerial BTSerial(10, 11); // RX = 10, TX = 11
void setup() {
  Serial.begin(9600);      // Serial Monitor
  BTSerial.begin(9600);    // Bluetooth default baud rate (HC-05)
  Serial.println("Bluetooth Module Test Ready...");
}
void loop() {
  if (BTSerial.available()) {
    char c = BTSerial.read();
    Serial.print("From Bluetooth: ");
    Serial.println(c);
  }
  if (Serial.available()) {
    char c = Serial.read();
    BTSerial.print(c);  // Send to Bluetooth
  }
}
```